03 — Debugging a Model That Runs but Does Not Learn

A step-by-step method for finding bugs that raise no errors, and a field guide to the ones you will meet most often.

Reading time: about 35 minutes. You need: modules 00 to 02. You do not need: any debugging tools beyond PyTorch itself.

Eight examples, and a loss that will not fall

Here is a real measurement from this module's exercise. A small transformer is trained on eight examples for 300 steps. A correct model memorises them easily: its loss falls from 2.777 to 0.013.

The broken model in the exercise does not. Its loss stops at 1.875 and stays there. Nothing crashes and no warning appears. The training loop runs to the end and reports a loss that looks almost reasonable.

That model contains eight bugs. None of them raises an error, and most are invisible when you read the code. A softmax over the wrong axis looks exactly like a softmax over the right one.

This is the situation in a debugging interview, and in everyday work. The common reaction is to reread the code, looking for something that seems wrong. That rarely works. What works is the opposite: predict a number, measure it, and let the difference point you to the bug.

What you will learn

By the end of this article, you will be able to:

- run the diagnostic ladder: six cheap checks, each of which rules out a whole class of bugs;
- recognise the most common silent bugs from their symptoms;
- trace a NaN back to the operation that produced it;
- work through a debugging interview in a way that shows your method.

Section 1 is the ladder. Section 2 is the field guide, section 3 covers NaNs, and section 4 covers the interview itself. A recap and the exercise briefing follow at the end.

---

1. The diagnostic ladder

Work from the top down. Each rung takes a few minutes, and each one rules out a large set of possible causes.

Rung 1: can the model overfit one small batch?

Take eight examples. Turn off dropout and weight decay. Then train on those eight examples only, for a few hundred steps.

A correct model with a correct training loop drives the loss close to zero. It has far more parameters than it needs to memorise eight examples. If the loss does not approach zero, there is a bug. And that bug is not in your data pipeline, your regularisation or your hyperparameters, because none of those matter on eight examples.

The shape of the loss curve tells you more than its final value. Here are three runs on the same eight examples:

Run	Step 0	Step 25	Step 100	Step 299
correct	2.777	0.214	0.017	0.013
`zero_grad()` missing	2.777	1.644	4.608	4.965
targets equal the inputs	2.811	0.112	0.007	0.002

Each row tells a different story:

- The correct run falls steadily towards zero.
- Without `zero_grad()`, the gradients from every step pile up. The loss falls briefly, then climbs, while the gradient norm grows from 2.2 to 141.
- When the targets accidentally equal the inputs, the loss falls *faster* than in the correct run. The model has learned to copy its input, which is useless. A loss that drops too easily is as suspicious as one that does not drop at all.

Rung 2: is the starting loss what it should be?

Before training, a model knows nothing, so it should give every class about the same probability. For V classes, the cross-entropy loss is then about $\ln V$. The exercise uses a vocabulary of 16 tokens, so its starting loss should be $\ln 16 = 2.773$. For GPT-2's 50,257 tokens it would be about 10.8.

A starting loss far from $\ln V$ is a useful clue:

- Much higher means the output layer starts too large, so the model is confidently wrong. Scale down that layer's initial weights.
- Much lower means the labels are leaking into the inputs, or are not what you think they are.
- A loss that starts at $\ln V$ and stays there means nothing is learning at all.

One warning: a correct starting loss proves little. The exercise's buggy model starts at exactly 2.773, despite its eight bugs.

Rung 3: does every parameter get a gradient, and does the optimiser see them all?

Run one backward pass, then look for parameters without a useful gradient:

```
for name, p in model.named_parameters():
    if p.grad is None:
        print("NO GRADIENT:", name)
    elif p.grad.norm() == 0:
        print("ZERO GRADIENT:", name)
```

The two messages mean different things. `None` means the parameter never took part in the computation. A norm of zero means it did, but the path to it is dead, for example because of a `.detach()` in the wrong place.

This check has a blind spot, and the exercise exposes it. `named_parameters()` only lists parameters that the model has registered. If some layers sit in a plain Python list, they are never registered, so the loop above finds nothing to complain about.

So also count the parameters and compare the total with what you expect. The exercise's buggy model reports 3,200 parameters; the correct model has 102,656. And check that the optimiser holds the same parameters as the model:

```
model_ids = {id(p) for p in model.parameters()}
opt_ids = {id(p) for g in opt.param_groups for p in g["params"]}
assert model_ids == opt_ids
```

Rung 4: are the gradients and activations a healthy size?

Log the gradient norm of every layer. Two patterns deserve attention:

- Gradients that shrink steadily from the output towards the input suggest a broken residual connection or an activation function that has saturated.
- One layer with a much larger gradient than the rest often points to a bad initialisation or a bad loss scale.

A well-known rule of thumb, from Stanford's CS231n course, compares each update with the weights it changes. The size of the update should be roughly a thousandth of the size of the weights. Much smaller, and the layer is barely learning. Much larger, and training is unstable.

Check activation sizes in the same way. Suppose each block makes its outputs three times larger. After ten blocks they are $3^{10} = 59{,}049$ times larger, which is close to float16's largest value of 65,504. One more block overflows.

Rung 5: test invariants, using shapes that are all different

Shape bugs that happen to broadcast survive every check above. Three defences catch them:

- Assert shapes where functions start and end, such as `assert logits.shape == (B, T, V)`.
- Test with small sizes that are all different, such as `B=2, T=5, H=3, D=7`. If every size is the same, a swapped axis is invisible.
- Test invariants rather than exact values. In a causal model, changing the last token must not change any earlier output. Attention weights must add up to one along the key axis.

The exercise's buggy model fails both invariants. Its attention weights miss summing to one by as much as 0.94. And changing a future token moves earlier outputs by up to 0.44.

Rung 6: compare with a reference implementation

When PyTorch provides the same layer, run yours and PyTorch's on the same input, and compare. For a whole model, compare intermediate outputs with a known-good implementation, layer by layer. Bisecting like this finds the first layer that differs within about $\log_2 L$ comparisons, for a model with L layers.

> **In the interview:** open with "before I read the code, let me check whether it can overfit one batch". It shows method straight away, and it rules out half the possible causes in a minute.

---

2. A field guide to silent bugs

These are the silent bugs that turn up most often in practice. The middle column describes what you will see. The last column explains why nothing raises an error.

Bug	What you see	Why nothing complains
`zero_grad()` missing	loss falls, then climbs; gradient norm grows every step	gradients keep adding up
softmax over the wrong axis	attention weights do not sum to 1; loss plateaus	`softmax(dim=-2)` on a square matrix is valid
mask applied after the softmax	weights sum to less than 1; outputs shrink	multiplying by a 0/1 mask is legal
mask or bias with the wrong shape	the mask or bias has no effect (see module 00)	many wrong shapes still broadcast
layers kept in a Python list	those layers never change; parameter count too low	the forward pass still uses them
softmax before `cross_entropy`	loss stuck at a floor, even on eight examples	`cross_entropy` accepts any numbers
targets not shifted by one	loss falls unusually fast; generated text is nonsense	predicting the current token is easy
attention heads split in the wrong order	loss plateaus high; causality breaks	both reshapes run
`model.eval()` never called	validation results are noisy and too pessimistic	dropout stays switched on
losses stored without `.item()`	memory grows until it runs out	each stored loss keeps its whole graph
learning rate too high	loss spikes to NaN, or stalls immediately	it looks like a modelling problem
`sum` instead of `mean` in the loss	effective learning rate grows with batch size	it looks like a tuning problem
data not shuffled	loss rises and falls in step with the epochs	training is valid, only badly ordered

The floor from the double softmax is easy to measure. In a model with every other bug fixed, the eight-example test stalled at 1.88 and would not go lower. Squashing the scores through a softmax twice limits how confident the model can ever become.

---

3. Tracing a NaN

A **NaN** ("not a number") is what floating-point arithmetic produces when a result is undefined. Once one appears, it spreads through every calculation it touches. NaNs have only a few causes, so a fixed routine finds them.

Here are the usual suspects, each checked in PyTorch:

Operation	Result
softmax of a row where every entry is $-\infty$	`[nan, nan, nan]`
`log(0)`	`-inf`
gradient of `sqrt` at 0	`inf`
`(-2) ** 0.5`	`nan`
`300 ** 2` in float16	`inf`

An infinity is only one step away from a NaN: $\infty - \infty$ and $0 \times \infty$ are both NaN. So work through these four steps in order:

1. Turn on anomaly detection with `torch.autograd.set_detect_anomaly(True)`. It slows training considerably, but it names the operation that first produced a NaN during the backward pass.
2. Look for log 0 and 0/0. Typical sources are a softmax over a fully masked row, the log of a probability that rounded to zero, and division by the length of an all-zero padding vector.
3. Look for overflow. float16 cannot store values above 65,504, and a long sum or an unscaled attention score can exceed that. Switching to bfloat16, or using loss scaling, fixes it.
4. Check the data. One corrupt example or an out-of-range token index is enough. An `assert not torch.isnan(x).any()` where data enters the model costs almost nothing.

If the forward pass is finite but the backward pass is not, look for functions with an infinite slope at a single point. `sqrt` at 0 is the classic case. PyTorch's built-in `norm` is protected at the zero vector and returns a gradient of 0 there. A norm written by hand as `x.pow(2).sum().sqrt()` is not protected: its gradient at zero is $0 \times \infty$, which is NaN.

---

4. How to work through a debugging interview

The ladder is also a script for the interview. Four habits make your method visible:

- **Narrate the ladder.** Say "first I will try to overfit one batch; if that fails, the data pipeline cannot be the problem". Saying this alone sets you apart.
- **Write a test before a fix.** When you suspect the mask, write the causality test first and watch it fail. Then fix the mask and watch it pass. Now you know the fix worked.
- **Change one thing at a time, and measure again.** If you change three things and results improve, you have learned nothing about which change mattered. Interviewers plant several bugs precisely to see whether you work this way.
- **Say what you would check next.** Finishing with five bugs found and a ranked list of three more hypotheses beats finishing with six bugs and no explanation.

Recap

- Measure instead of reading. Silent bugs produce valid tensors of plausible size.
- Start with the overfit test on one small batch. Look at the shape of the curve, not only its end.
- Check the starting loss against $\ln V$, count parameters, and confirm the optimiser holds them.
- Test invariants such as causality, using shapes that are all different.
- Trace NaNs with anomaly detection, then look for log 0, 0/0, overflow and bad data.
- In an interview, narrate the ladder and write a test before each fix.

The exercise: find eight planted bugs

What you will do. This exercise is about fixing, not building. The file `buggy_lm.py` contains a two-layer transformer language model trained on the copy task from module 02. It contains eight planted bugs, all drawn from the field guide in section 2. Your job is to find and fix all eight.

The notebook gives you the diagnostic ladder as ready-made tools, plus nine checks, labelled A to I, that pass only when the model is correct.

Time and setup. Aim to find all eight bugs within an hour; allow up to two. The setup is the same as for module 00.

How to run it. Open `03-ml-debugging/exercise.ipynb` and run all the cells. You will edit `buggy_lm.py` in a separate editor. After each edit, run the notebook's `reload()` cell so it picks up your change.

Three working rules.

1. Do not read `buggy_lm.py` hunting for bugs. Run the diagnostics, and let a failing measurement tell you where to look.
2. Fix one bug at a time, then run the checks again.
3. Before each fix, make sure some check fails because of that bug. Afterwards, you will know the fix worked.

How the checks work. The first run shows the starting state. Long lines are shortened here:

```
model: 3200 parameters
[FAIL] A  targets are the NEXT token  -- y[t] should equal x[t+1]
[FAIL] B  every submodule is registered  -- orphaned: ['blocks[0]', ...
[FAIL] C  the optimiser would see the whole model  -- 3200 parameters ...
[ok]   D  every registered parameter gets a nonzero gradient
[FAIL] E  attention weights sum to 1 over keys  -- worst deviation 9.443e-01
[FAIL] F  the future cannot leak into the past  -- max leakage 4.351e-01
[ok]   G  initial loss is ln(V)  -- 2.773 vs ln(16) = 2.773
[FAIL] H  evaluate() runs with dropout off and no graph  -- 0 dropout ...
[FAIL] I  can overfit a single batch  -- final loss 1.87506
```

Notice that checks D and G pass, even though the model is broken. Section 1 explained why: D cannot see layers that were never registered, and a correct starting loss proves little.

Check map. This table links each check to the rung of the ladder it comes from.

Check	What it tests	Where it comes from
A	targets are the next token	section 2, unshifted targets
B, C	every layer is registered and visible	rung 3
D	every registered parameter gets a gradient	rung 3
E	attention weights sum to one	rung 5
F	the future cannot affect the past	rung 5
G	the starting loss is ln V	rung 2
H	evaluation runs with dropout off and no graph	section 2, `eval()` missing
I	the model can overfit one batch	rung 1

When you are done. All nine checks pass. The last cell then trains the fixed model, which predicts the second half of the copy task with 100% accuracy.

Its validation loss settles near 1.27, not near zero, for the same reason as in module 02. Of the 15 predicted positions, 7 fall in the random first half, where each token is one of 15 equally likely values. So the best possible loss is $\frac{7}{15} \ln 15 = 1.264$. The reference model reaches 1.266.

Then write a two-line note for each bug: what it was, which measurement exposed it, and why it raised no error. Compare your notes with the annotated list at the top of `solution.py`. Compare your *route* as well as your answers: did each bug come from a measurement, or from reading?

If you get stuck.

- Work through the notebook's guided sections in order. They start with the cheapest signals.
- Print the attention weights for one head. A correct causal pattern is lower-triangular, with rows that sum to one.
- For the head-split bug, perturb one input position and see which outputs move.
- Plot the overfit curve rather than reading its final number. The accumulation bug and the double-softmax bug look completely different.

Want more? The notebook ends with three stretch goals. You can plant a ninth bug for someone else to find, or practise the NaN routine on a model you break on purpose. You can also add a shape-assertion helper throughout `buggy_lm.py`.

Going deeper

Start here

- Karpathy (2019), *A Recipe for Training Neural Networks* — https://karpathy.github.io/2019/04/25/recipe/. The best single document on this subject, and the source of the overfit-one-batch habit. Written from experience, with many specific failure stories.

Checks and heuristics

- Stanford CS231n, *Neural Networks Part 3: Learning and Evaluation* — https://cs231n.github.io/neural-networks-3/. Covers gradient checking, the update-to-weight ratio from rung 4, and other sanity checks.
- PyTorch, *Autograd anomaly detection* — https://pytorch.org/docs/stable/autograd.html#anomaly-detection. How `set_detect_anomaly` works, and what it costs.
- Bengio (2012), *Practical Recommendations for Gradient-Based Training of Deep Architectures*. arXiv:1206.5533. Older, but still correct on learning rates and on what to check first.
- Karpathy (2018), a widely shared list of common neural-network mistakes — https://twitter.com/karpathy/status/1013244313327681536. A short, memorable version of the field guide in section 2.

Why these methods work

- Zhang et al. (2017), *Understanding Deep Learning Requires Rethinking Generalization*, ICLR. arXiv:1611.03530. Shows that networks can memorise even random labels, which is why the overfit test is a fair expectation.
- Micikevicius et al. (2018), *Mixed Precision Training*, ICLR. arXiv:1710.03740. Section 3 explains loss scaling, the main defence against float16 overflow.

Debugging in production

- Sculley et al. (2015), *Hidden Technical Debt in Machine Learning Systems*, NeurIPS. Why machine-learning systems fail silently at the organisational level too.
- Huyen (2022), *Designing Machine Learning Systems*, O'Reilly. Chapter 6 covers model development and debugging in real products.

Next: module 04 assumes the model is correct, and asks how to fit its training onto real hardware.